Project Report: Flexclone Implementation in Ext4

April 26, 2026

Group Details

- **Member 1:** Aarsh Jain — 230015 — aarshjain23@iitk.ac.in
- **Member 2:** Surepally Pranay Sriharsha — 231057 — surepally23@iitk.ac.in
- **Member 3:** Saksham Verma — 230899 — sakshamv23@iitk.ac.in
- **Member 4:** Shlok Jain — 230493 — jainshlok23@iitk.ac.in
- **Member 5:** Yatharth Sharma — 231198 — yatharths23@iitk.ac.in
- **Supervisor:** Rohit Singh — CSE
- **Supervisor:** Debadatta Mishra (Prof.) — CSE

Contents

1 Abstract **3**

2 Introduction **3**

3 Design and Architecture **4**

4 Implementation Details **6**

5 Evaluation and Results **8**

6 Conclusion and Future Work **11**

7 Testing Philosophy and Instructions **12**

7.1 Hello World Test: 12

7.2 Detailed Evaluation 12

7.3 Experiments 13

1 Abstract

This project implements a versioned, centralized storage architecture within the **Flexclone** filesystem framework, designed for high-efficiency file cloning and strict data consistency. The system uses a parent-centric model in which all physical data is consolidated into a single parent file, while Flexclone child files serve as lightweight, metadata-only pointers. To guarantee crash consistency and prevent data corruption during concurrent operations, we integrated custom **transaction semantics and CRC-based verification into Flexclone natively**. Utilizing a specialized ioctl framework to define strict transaction bounds, the system enforces inode-level mutual exclusion and atomic write commits, ensuring that intermediate states are never exposed to child views.

At its core, the storage architecture relies on an efficient append-only strategy. Instead of overwriting existing data, any modifications to the parent file are simply appended to the end. A dedicated log file keeps track of this history, mapping logical block numbers and version IDs to their actual physical locations on the disk.

Flexclone child files act as lightweight, read-only snapshots tied to specific versions, dynamically building their state by reading this log. When needed, a commit operation allows a child to instantly sync up with the parent's most recent state. To protect against system crashes or sudden failures, the transactional design automatically rolls back any incomplete updates to the last safe, consistent version. Finally, a built-in garbage collector works behind the scenes to find and free up physical blocks that are no longer linked to any active Flexclone version. This creates a highly reliable, high-performance system for fast file cloning and point-in-time recovery.

2 Introduction

- **Background & Motivation:** The flexclone filesystem architecture offers a highly efficient approach to file cloning by centralizing physical data within a single parent file, while child files act as lightweight, metadata-driven pointers. While effective for space optimization, the original flexclone implementation was fundamentally limited by its lack of multi-block atomicity, which guaranteed data consistency only at the individual block level. When applications issued multi-block write operations, there was no overarching transactional boundary grouping the modifications. If a system crash or power failure occurred mid-operation, the filesystem was highly susceptible to "torn writes," permanently leaving the storage in a corrupted, intermediate state. Furthermore, this lack of operational isolation led to "torn reads," a scenario where child files querying the parent during an active update could retrieve an inconsistent mix of obsolete and newly modified blocks. Overcoming these concurrency and crash-consistency vulnerabilities is essential to making the architecture reliable.

- **Problem Statement:** The objective of this project is to design and implement strict transaction semantics natively within the flexclone filesystem to solve the inherent consistency flaws of the original design. The upgraded architecture must group multi-block parent updates into atomic, all-or-nothing operations to completely eliminate the risk of torn writes during system crashes. Additionally, the system must enforce proper read isolation, ensuring that child files access only fully committed, consistent versions of the parent data, thereby preventing torn reads. The core technical challenge lies in engineering a custom, ioctl-driven transactional locking and logging framework that guarantees this atomicity and mutual exclusion without introducing severe performance bottlenecks or excessive disk I/O overhead.

- **Contributions:** The primary deliverable of this project is a specialized Linux kernel

Ext4-module that introduces a centralized, versioned cloning architecture. The core contributions include:

- **Atomic Transaction Framework:** Implemented ioctl-driven transaction boundaries to guarantee multi-block atomicity and mutual exclusion, completely eliminating the torn write and torn read vulnerabilities of the original design.
- **Versioned "See-Thru" Reads:** Upgraded the child file mechanism to dynamically query an append-only transaction log. This ensures child instances always serve as isolated, crash-consistent views tied to specific parent transaction states.
- **Deterministic Synchronization:** Introduced custom interfaces that allow child files to explicitly "commit" to the parent's timeline, enabling instant, point-in-time metadata synchronization.
- **Space-Efficient Versioning:** Integrated a supplementary block eviction (garbage collection) mechanism to reclaim orphaned physical blocks, ensuring the new append-only tracking model remains storage-efficient over time.

3 Design and Architecture

- **The Flexclone Model:** FlexClone is a file cloning system designed to maintain data consistency when a file is updated while its clones are being accessed simultaneously. In standard cloning, a "mixed block" problem often occurs, where a reader sees an inconsistent blend of old and new data because the source changes during the read operation. FlexClone solves this by using versioning and a transactional log to create a stable, consistent view of the file. By ensuring that new writes do not overwrite data currently needed by clones, the system allows read requests to pull from a specific, finalized state, guaranteeing that readers see a reliable version even during active updates.

- **The Parent File Stores All Data:** All actual data is stored in a single Parent File. This file holds the original "base" data and all subsequent updates. When information is updated, the new version is appended to the parent file rather than overwriting the original blocks. This includes a See-Thru Read-Only (RO) mode, which allows the parent to be updated while keeping the old data safe for clones that still need to read it.

- **See-Thru Read-Only Redirection:** When a block range is set to `see_thru_ro` mode, the system automatically redirects all access to the parent file. In this mode, the child is permitted to read the data but is strictly blocked from making any changes to those specific blocks.

- **The Redirect System:** When a user reads a clone, the system intercepts the request and identifies which version the clone requires. It uses an in-memory map (an XArray) to find the exact location of that data within the parent file and pulls it from there.

- **Data Structures:** The system bridges persistent storage and high-performance memory lookups using two highly optimized primary data structures:

  - **On-Disk Transaction Log (scorw_log_record):** The system maintains an append-only binary log tracking all block modifications. Each state change is recorded using a strictly aligned 32-byte struct containing the version number, logical start block, physical start block, and the contiguous length of the write (`len_blks`). Crucially, this struct utilizes explicit padding to guarantee a 32-byte size. This mathematical alignment ensures exactly 128 records fit into a standard 4 KB Ext4 block, physically preventing any metadata record from straddling a page boundary, which eliminates a critical vector for memory faults and torn reads.

- **In-Memory Index Engine (XArray):** To prevent severe disk I/O latency during "See-Thru" read operations, the system does not parse the log file synchronously on every `read()` syscall. Instead, it utilizes the Linux kernel's native XArray data structure. We maintain an array of `struct scorw_version` in the parent `scorw_inode`, where each `scorw_version` contains a version number and a radix tree (Xarray) containing the metadata for all the modified blocks in that version. It stores the parent file index for each modified block in that version. *(P.S. at any given moment, only MAX_VERSION is allowed in memory; we follow a FIFO scheme for eviction)*
- **Transaction locks:** The inode maintains a `transaction_locks` struct, which stores three things: a mutex lock to ensure mutual exclusion during transactions, a pointer to the owning file to prevent other processes from taking the lock, since they will have different file pointers even though the inode is the same. The third is a variable 'transaction', which lets us know what state we are in: `SET_TRANSACTION`, `SET_NORMAL_WRITE`, or none.

- **Multi-block Write Consistency:** Standard See-Thru RO implementations are vulnerable to "torn writes" during multi-block operations if a hardware failure interrupts the sequence. This architecture guarantees strict atomicity and consistency across multiple blocks via a custom transaction framework natively integrated into the VFS.

- **Strict Transaction Boundaries:** Multi-block writes are explicitly bounded by `BEGIN_TRANSACTION` and `END_TRANSACTION` ioctl calls invoked from user space. This transforms a series of independent `sys_write` operations into a single, cohesive logical unit.

- **Write Isolation and Mutual Exclusion:** When a transaction begins, the module enforces inode-level mutual exclusion using kernel locking primitives (e.g., `down_write` on the inode's semaphore). During the active transaction, new data blocks are physically appended to the parent file, and their corresponding 32-byte tracking structures are appended to the log file. Because the transaction is not yet committed, these physical writes remain completely isolated and invisible to any child files querying the parent.

- **Atomic Commits and Crash Recovery:** The state transition becomes visible only when the `END_TRANSACTION` boundary is executed, serving as the atomic commit marker. If a sudden power loss or kernel panic occurs mid-transaction, the system recovers deterministically during the next mount phase. The mount routine scans the log, detects the absence of a finalized commit marker for the interrupted operation, and automatically truncates the uncommitted log entries. This "roll-forward, rollback" mechanism guarantees that the system always boots into the last known-good transactional state, mathematically precluding torn writes or corrupted physical mappings.

4 Implementation Details

1. **Metadata Linkage and Initialization (`setexattr`)** The structural relationship between the parent, child (shadow), and log files is established using extended attributes. When `setexattr` is invoked, the system creates a bi-directional reference network:
   - The log file's inode is stored within the parent file's extended attributes.
   - The parent file's inode is stored within the child file's extended attributes. This ensures all associated components—parent, child, and log—remain securely linked and locatable during file operations.
2. **Transactional Write Operations** All write operations to the parent file are strictly isolated using a transactional model to ensure consistency. The write lifecycle proceeds as follows:
   - **Initiation:** The operation begins with a `BEGIN_TRANSACTION` call, which acquires the necessary kernel locks to enforce mutual exclusion.
   - **Execution:** Modified data blocks are physically appended to the end of the parent file. Simultaneously, a corresponding mapping entry is recorded in the log file. The version number that tracks the file state is then updated in the parent's extended attributes.
   - **Completion:** The operation concludes with an `END_TRANSACTION` call, which commits the changes and releases the locks.
3. **Transaction Commit and Crash Recovery** The system outlines two distinct versions for handling the `END_TRANSACTION` commit phase and crash consistency:
   - **Version 1 (In-Memory Synchronization):** Upon commit, only the log file is explicitly synchronized to the disk. The structural updates to the parent file and the incremented version number are updated solely in memory.
   - **Version 2 (CRC-Based Recovery):** To protect against kernel crashes, a Cyclic Redundancy Check (CRC) is computed using the parent file, and this CRC is also stored in the log for each entry.
4. **Versioned Read Operations** Read requests are dynamically routed based on the versioning data stored in the log file, allowing parent and child files to access different states of the data.
   - **Parent Read:** The system queries the log file using the parent's current version number to determine the correct physical block offset. It then executes a generic read on the parent file using this retrieved offset.
   - **Child Read:** Read requests directed at the child file are redirected to the parent file's read mechanism. The system uses the child's specific version number to fetch the corresponding historical offset from the log file. A generic read is then performed on the parent file at that specific offset, ensuring the child accesses its correct, isolated snapshot.
   - **Log file querying:** Since different children and parents can be at different versions, when reads are performed from these files, we need to have log file structures stored in memory, but since storing the whole log file in memory is very inefficient in terms of memory, we implement an on-demand version loading and eviction system, which works as follows.
     - For each parent file, you can have a maximum of 32 version xarray structures in memory.

- If there are 32 versions already and a new version is requested (by reading any file), a previously stored version is evicted from memory on a **FIFO (first in, first out)** basis, and the new version is loaded.
- To load a new version, the log file is not fully read; instead, it is read efficiently.
- Since the log file stores versions in **increasing order**, a **binary search** is performed to find the start and end offsets of a particular version, and then reads are performed to load that version into memory.

5. **Post-Mount Initialization and Eviction** When the parent file is opened for the first time following a filesystem mount, an initialization sequence is triggered to ensure consistency across the components.

   Based on the Version 1 implementation, this involves three steps:

   - **Attribute Sync:** The log file state is synchronized with the extended attribute data present on the parent inode.
   - **Version Alignment:** The log file and the physical parent file are synchronized to guarantee both entities reflect the identical, latest version state.
   - **Block Eviction:** A cleanup routine identifies and removes unused or orphaned data blocks from the system.

   Based on the Version 2 implementation, this involved these steps:

   - **Attribute Sync, Version Alignment, and Block Eviction** (same as Version 1).
   - **Incomplete version removal:** Since this version implementation doesn't explicitly sync the parent and log files at the end of the transaction, there is a chance that the log points to an offset where data isn't actually written. This will corrupt that entire version. To ensure that such versions are correctly removed and that the log, as well as parent files, are rolled back to the last safe version, a CRC check is performed. This calculates the CRC for versions after the last safe version, and if there is a mismatch between the CRC saved in the log file and its calculated value using the parent file, both the log and parent file are reverted back to the last previous consistent version.

5 Evaluation and Results

- **4.1 Experimental Setup:** The evaluation was conducted on a Linux Virtual Machine environment configured with a custom kernel (version 96.12.43, built from the provided maha-flex-clone-cs-614-2026-Maha_port_clone source tree). The custom FlexClone implementation is deployed as a modified ext4 kernel module, leveraging custom ioctl calls and extended attributes (fsetxattr) to manage snapshot metadata.
  - **Hardware:** [4 vCPUs, 4GB RAM, 50GB virtual disk + 15GB partition for testing Flexclone fs + 5GB Swap]
  - The exact config file and other changes made to the Linux kernel are present in [Version1(Log sync Version)] and [Version2(No sync Version)]
- **4.2 Correctness Testing:** To ensure the reliability and structural integrity of the file system under our new FlexClone implementation, we validated the system against race conditions and consistency errors, with a primary focus on the novel see-thru functionality:
  - **See-Thru Consistency:** We validated that updates made in the parent file are consistently visible to the child file within the specified block ranges. To ensure synchronization, we explicitly issue an update-child ioctl command, which updates the child's internal version to match the latest parent version. This mechanism was verified by executing targeted write operations on the parent using our custom test suite (e.g., writing 4096-byte blocks at specific offsets using write_4096.c), invoking the ioctl, and subsequently reading from the cloned child to assert that the exact modifications were reflected without data corruption.
  - **Race Conditions & Concurrency:** We evaluated the system's resilience to race conditions by initiating concurrent read/write workloads. Specifically, we tested scenarios in which the parent file was being written to by two processes simultaneously. All this was handled by kernel-level locking mechanisms used for both normal and transactional writes.
  - **Edge Case Testing:** We employed a dedicated testing suite (test_edge_cases.c) to maliciously stress the file system. We simulated protocol violations to ensure the kernel module safely rejects invalid operations without crashing or leaving metadata in an inconsistent state.
  - **Transaction Atomicity:** We deliberately avoided relying on standard journaling and instead built a custom, lightweight transaction mechanism (scorw_set_transaction). This uses a dedicated transaction_lock on the scorw_inode to guarantee mutual exclusion per file descriptor. By explicitly wrapping operations in SET_TRANSACTION and UNSET_TRANSACTION blocks, we ensure that changes are staged safely. Upon UNSET_TRANSACTION, the system forces a flush of both the file data (ext4_sync_file) and our custom log metadata, and only then atomically increments the clone's version number to commit the transaction.
- **4.3 Performance Benchmarks:** We evaluated the performance impact of our custom enhancements, measuring the time taken for write operations across different file system configurations.
  - **Write Performance Comparison:** We compared the baseline performance of a 10MB write operation on a 1GB file across three configurations:
    - **Baseline Ext4:** 0.015 seconds
    - **Old Flexclone:** 0.021 seconds
    - **New Flexclone (See-Thru):** 0.028 seconds

For 500MB write operations the timings are as follows.

- **Baseline Ext4:** 0.5 seconds (Averaged over 3 runs)
- **Old Flexclone:** 0.5 seconds (Averaged over 3 runs)
- **New Flexclone:** 0.8 seconds (Averaged over 3 runs)

The New Flexclone introduces an expected overhead due to the metadata management and block-range redirection required for the see-thru functionality.

- **Transactional Write Performance:** We further benchmarked the performance of our custom transaction semantics within the New Flexclone. The timing for write operations using explicit transaction blocks on a 1GB file was recorded as follows:
  - **10MB Write:** 0.03 seconds (Averaged over 4 writes)
  - **New Flexclone (Log file sync version):** 0.9 seconds (Averaged over 3 runs)
  - **New Flexclone (No sync version):** 0.8 seconds (Averaged over 3 runs)

  These results demonstrate the scalability and consistency overhead of the transaction locking mechanism and subsequent metadata log syncing during see-thru updates.

- **4.4 Throughput Comparison:** The below graph shows throughput of native-ext4 vs modified Flexclone-fs

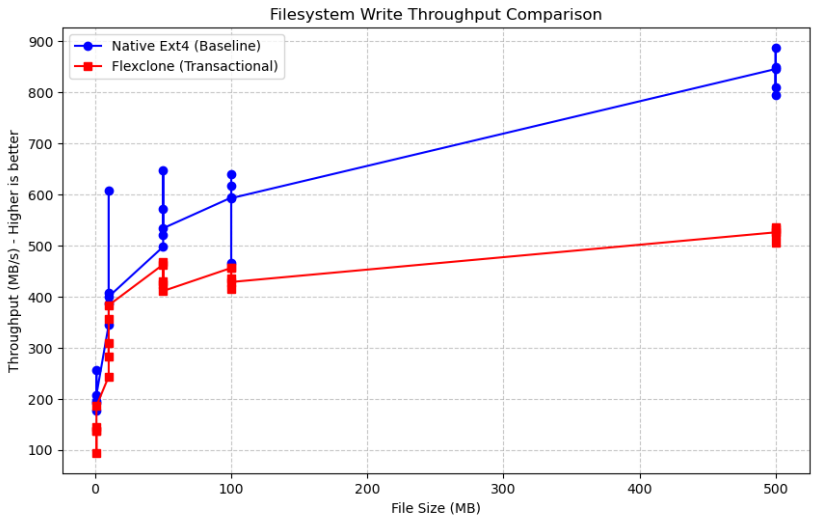


Figure 1: Throughput comparison between Native Ext4 and Modified Flexclone-fs

- **4.5 Phase-wise Timing Analysis:** We compare the average time spent in each phase of the transactional write path for two implementations: the **Log Sync Version (Version1)** and the **No Sync Version (Version2)**. Note that the Fsync is done externally after ENDtxn.

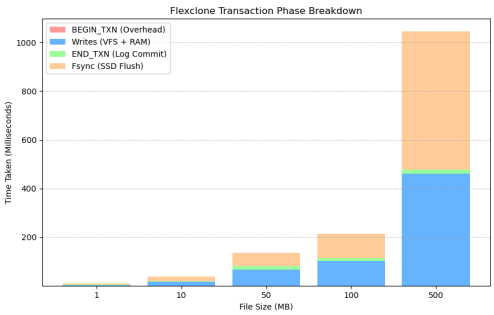
(a) Log Sync Version (Version1)

MB	Begin	Write	End	Fsync
1	0.0020	1.8310	3.1028	4.5448
10	0.0036	16.7742	2.8532	18.0078
50	0.0022	67.6176	13.3034	53.7640
100	0.0022	102.5436	11.9710	99.9100
500	0.0060	462.0932	16.4206	567.2578

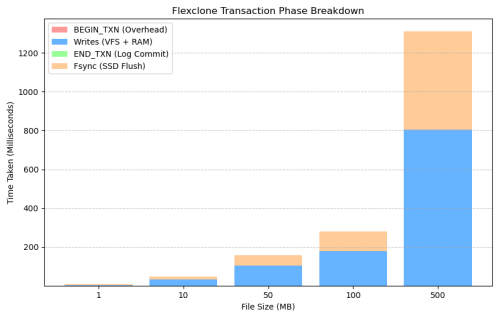
(b) No Sync Version (Version2)

MB	Begin	Write	End	Fsync
1	0.0022	3.5040	0.0012	5.1090
10	0.0022	33.5980	0.0030	15.2496
50	0.0022	104.2524	0.0022	54.6866
100	0.0028	180.2248	0.0022	98.8392
500	0.0022	803.5212	0.0022	507.3304

Table 1: Average timings per phase (milliseconds)



(a) Log Sync Version (Version1)



(b) No Sync Version (Version2)

Figure 2: Phase-wise timing plots for both implementations

From Table 1 and Figure 2, both versions incur negligible Begin overhead. Version1 spends more time in the End phase due to log file sync, while Version2 minimizes this cost. For larger workloads, Version2 shifts more overhead into the Write phase (Calculating CRC), whereas Version1 distributes the cost between writing and transaction completion.

Overall, Version1 favors stronger persistence guarantees through immediate synchronization, while Version2 improves responsiveness by reducing sync-related latency.

6 Conclusion and Future Work

- **Summary:** We have successfully implemented two variations for the project. Each variation was developed to address distinct requirements while maintaining a consistent overall architecture. Both implementations underwent rigorous testing, including functional, integration, and edge-case validation. The results confirm that the two variations perform reliably and meet the defined acceptance criteria.

- **Limitations & Future Optimizations:**
  - In the current system, a single file cannot use multiple flexclone modes across different ranges, but this capability could be added.
  - Garbage collection (eviction) is currently implemented by simply punching holes in the parent file, which turns it into a sparse file and introduces its own drawbacks. Instead, this could be modified so that, at regular time intervals, a background process compacts the sparse file back into a dense file and updates the log file accordingly.

7 Testing Philosophy and Instructions

We have tested our system extensively on a diverse range of scenarios, including basic tests for versioning, testing using large files of 1GB-2GB, testing to evaluate performance in terms of timings of operations, testing kernel panics, accidental shutdowns, to ensure crash consistency, to evaluate if the garbage collection mechanism is working, is the CRC based system able to detect inc consistencies across various file sizes ranging from few MBs to a few GBs, etc.

7.1 Hello World Test:

For basic verification of how the system works, do the following steps:

1. Insert the module using: `./insert_ext4_module.sh`
2. Mount a fresh device using: `sudo mount -t ext4-module /dev/sda4 /mnt`
3. Create a parent file, and write anything to it.
4. Establish the child, and log file relations using: `./setxattr_generic -p par -c child -f frnd -l logf -r 0:3:3`
   Where the last argument should be like 0:¡size of parent in terms of number of blocks¿:3
5. Now you can write to any offset of the parent file using: `./write_offset <par> <offset> <content>`
6. Now, a read on the parent file will return the updated contents, but the child file will still have the previous version.
7. To update the child to the latest parent version, use this: `./update_child <child>`
8. Now, the child would have been updated to the latest version.

7.2 Detailed Evaluation

- **Analysis of Version 1:** Any crash before the log file completely syncs is easily detected as the version is incremented only after the log file syncs. This is detected at the first opening of the parent file, and the log file is corrected to the previous version. Therefore, we have the guarantee that the log file is consistent with the version at mount time (either all records of a version or none).

  And now the following loop is run.

  ```
  for(v = latest_log_version_found ; v >=0 ; v = v-1){
      if(latest_parent_logical_offset_in_version_v <= parent(size)){
          Truncate the parent and log till this point;
          return;
      }
  }
  ```

  For example, consider this: a parent synced to version 4, with a few blocks from version 5 synced. Now, if the log file shows a version < 5, then the parent file will be truncated to match the log file's version (the first iteration in the for loop) Else if the log file shows a version > 4, then the log file and its parent will be truncated to show a consistent view up to version 4.

- **Analysis of Version 2:** Since the log and parent files aren't explicitly synced to disk, there is a chance the log file points to an offset where data isn't actually written. To revert to such versions, a CRC check is performed between the CRC stored in the log file and the calculated CRC. If a mismatch occurs, the version is dropped, and both the log and parent files are rolled back to the previous consistent version.

7.3 Experiments

Experiment 1: recovery_test1.sh

- **Purpose of the experiment:** Verifies the recovery done, while opening the parent file for the first time, for version 1 (Case when Parent is synced completely, and the log isn't)
- **What actually happens in the experiment:** We create a parent file of 10Blocks with flexclone-fs. We create 2 versions of the parent, then unmount the flexlone-fs and mount it with ext4. Now we append 2 new blocks at the end of the parent and remount again with flexclone-fs.
- **How to run the experiment:** `./recovery_test1.sh`
- **Estimated runtime:** Human-time = 0mins , Compute-time < 1min
- **Expected result:** Recovery of the last version of the parent created when mounted with flexclone-fs.(The extra appended blocks are deleted)
- **How to access the actual result:** Results are printed on the terminal/stdout

Experiment 2: fake_log_append_test.sh

- **Purpose of the experiment:** Verifies the recovery done, while opening the parent file for the first time, for version 1 (Case when log is synced completely and the parent is not)
- **What actually happens in the experiment:** We create a parent file of 10Blocks with flexclone-fs. We create 2 versions of the parent, then unmount the flexlone-fs and mount it with ext4. Now we append 2 fake-log records at the end of the parent and remount again with flexclone-fs.
- **How to run the experiment:** `./fake_log_append_test.sh`
- **Estimated runtime:** Human-time = 0mins , Compute-time < 1min
- **Expected result:** Recovery of the last version of the parent created when mounted with flexclone-fs.(The extra appended log records are deleted)
- **How to access the actual result:** Results are printed on the terminal/stdout

Experiment 3: time_analysis.sh

- **Purpose of the experiment:** Time analysis for different scenarios
- **What actually happens in the experiment:** We set up the required environment, then create a parent file of 1GB, and calculate write time for a 10MB write on it using Transactional semantics and without using it. We also calculate the write time of 500MB using Transactional semantics. (we repeatedly create new files and erase old ones, to avoid memory overheads)
- **How to run the experiment:** `./test2.sh`

- **Estimated runtime:** Human-time = 0mins, Compute-time < 1min
- **Expected result:** Time taken for different types of writes using Transaction semantics and without using Transaction semantics
- **How to access the actual result:** Results are printed on the terminal/stdout

Experiment 4: basic_test1.sh

- **Purpose of the experiment:** Verifying the correctness of the version control system, using a single child
- **What actually happens in the experiment:** Create a 10-block parent file and a single child file. Do some simple writes on the parent file, and then verify that the child is in the old version. Then call update child IOCTL, and verify that the child is updated to the latest parent version.
- **How to run the experiment:** `./basic_test1.sh`
- **Estimated runtime:** Human-time = 0mins, Compute-time < 1min
- **Expected result:** Child remains at the older version unless the update IOCTL is called
- **How to access the actual result:** Results are printed on the terminal/stdout

Experiment 5: basic_test2.sh

- **Purpose of the experiment:** Verifying the correctness of the version control system, using multiple children
- **What actually happens in the experiment:** Create a 10-block parent file and four child files. Do four simple writes on the parent file and update one unique child after each update. Now each child is in a different version.
- **How to run the experiment:** `./basic_test2.sh`
- **Estimated runtime:** Human-time = 0mins, Compute-time < 1min
- **Expected result:** Each child is at a different version of the parent file
- **How to access the actual result:** Results are printed on the terminal/stdout

Experiment 6: edge_case_testing1.sh

- **Purpose of the experiment:** Testing edge cases
- **What actually happens in the experiment:** Create a 10-block parent file and a single child file. And then doing the following -
  - Calling only START_TRANSACTION on the parent file
  - Calling only END_TRANSACTION on the parent file
  - Calling multiple START_TRANSACTION before closing the fd
  - Calling fork(), then simultaneously initiating a transaction from both
- **How to run the experiment:** `./edge_case_testing1.sh`
- **Estimated runtime:** Human-time = 0mins, Compute-time < 1min

- **Expected result:** Results per subtest (respectively) -
  - The lock is released on closing the fd
  - IOCTL call fails
  - Only the first START_TRANSACTION IOCTL is successful; the rest fail
  - As one process acquires the lock, the other Transaction IOCTL call fails
- **How to access the actual result:** Results are printed on the terminal/stdout

Experiment 7: poweroff_testing.sh

- **Purpose of the experiment:** Testing the Power-off scenario during the transaction
- **What actually happens in the experiment:** Create a 10-block parent file and a single child file. Start a transaction, write a few things, and before committing, we get stuck in an infinite loop.
- **How to run the experiment:** `./poweroff_testing.sh`, and then Force Shutdown the VM and restart it when requested, and finally re-setup the filesystem and verify the parent file.
- **Estimated runtime:** Human-time = 1-2mins, Compute-time < 1min
- **Expected result:** The last transaction changes should not be reflected, since we didn't commit them.
- **How to access the actual result:** Need to manually reboot, set up the environment, then check the parent file

Experiment 8: panic_testing1.sh

- **Purpose of the experiment:** Testing kernel panics during fsync
- **What actually happens in the experiment:** Create a 10-block parent file and a single child file. Set up a sysfs entry that triggers panic points in the kernel. Initiate a transaction. While committing the transaction, fsync is called, which triggers panic just before data syncing.
- **How to run the experiment:** `./panic_testing1.sh`, and then Force Shutdown the VM and restart it when requested, and finally re-setup the filesystem and verify the parent file.
- **Estimated runtime:** Human-time = 1-2mins, Compute-time < 1min
- **Expected result:** Last transaction changes should not be reflected, as we didn't commit them properly.
- **How to access the actual result:** Need to manually reboot, set up the environment, then check the parent file

Experiment 9: panic_testing2.sh

- **Purpose of the experiment:** Testing kernel panics during fsync
- **What actually happens in the experiment:** Create a 10-block parent file and a single child file. Set up a sysfs entry that triggers panic points in the kernel. Initiate a transaction. While committing the transaction, fsync is called, which triggers panic just before metadata syncing and just after data syncing.
- **How to run the experiment:** `./panic_testing2.sh`, and then Force Shutdown the VM and restart it when requested, and finally re-setup the filesystem and verify the parent file.
- **Estimated runtime:** Human-time = 1-2mins, Compute-time < 1min
- **Expected result:** Last transaction changes should not be reflected, as we didn't commit them properly.
- **How to access the actual result:** Need to manually reboot, set up the environment, then check the parent file

Experiment 10: test_crash_after_log_write.sh (For CRC version)

- **Purpose of the experiment:** Testing how the log file changes and how data in the parent file changes if a crash happens after the log file is written to disk and the parent file is not written to disk.
- **What actually happens in the experiment:** It creates a parent file named `test_par_file` with 2 blocks, a child file named `test_child_file`, a log file named `test_log_file`, and a friend file named `test_frnd_file`. It updates the parent file, sets the sysfs variable `scorw_crash_after_log_write`, and then writes to the parent again, which causes a crash. Now, reboot the system and again run `./run_after_crash.sh`, which prints the log file content and the parent file, which will not be affected by the crash that occurred during the last write.
- **How to run the experiment:** `sudo ./test_crash_after_log_write.sh`, and then Force Shutdown the VM and run `sudo ./run_after_crash.sh`.
- **Expected result:** The last transaction changes should not be reflected because a crash occurred during the transaction.

Experiment 11: test_crash_bad_crc.sh (For CRC version)

- **Purpose of the experiment:** Testing how the log file changes and how data in the parent file changes if a crash happens after the log file is written to disk and the parent file is not written to disk.
- **What actually happens in the experiment:** It creates a parent file named `test_par_file` with 2 blocks, a child file named `test_child_file`, a log file named `test_log_file`, and a friend file named `test_frnd_file`. It updates the parent file, sets the sysfs variable `scorw_crash_with_bad_crc`, and then writes to the parent again, which causes a crash. Now, reboot the system and again run `./run_after_crash.sh`, which prints the log file content and the parent file, which will not be affected by the crash that occurred during the last write.

- **How to run the experiment:** `sudo ./test_crash_bad_crc.sh`, and then Force Shutdown the VM and run `sudo ./run_after_crash.sh`.
- **Expected result:** The last transaction changes should not be reflected, since we manually changed the CRC to the wrong value.

Experiment 12: test_crash_after_par_write.sh (For CRC version)

- **Purpose of the experiment:** Testing how the log file changes and how data in the parent file changes if a crash happens after the parent file is written to disk and the log file is not written to disk.
- **What actually happens in the experiment:** It creates a parent file named `test_par_file` with 2 blocks, a child file named `test_child_file`, a log file named `test_log_file`, and a friend file named `test_frnd_file`. It updates the parent file, sets the sysfs variable `scorw_crash_after_data_write`, and then writes to the parent, which causes a crash. At this point, the parent file has been fsynced to disk, but the log file is not. Now, reboot the system and run `./run_after_crash.sh` again, which prints the contents of the log file and the parent file, which will not be affected by the crash that occurred during the last write.
- **How to run the experiment:** `sudo ./test_crash_after_data_write.sh`, and then Force Shutdown the VM and run `sudo ./run_after_crash.sh`.
- **Expected result:** The last transaction changes should not be reflected because a crash occurred during the transaction.

Experiment 13: Eviction testing

- **Purpose of the experiment:** To test whether an unused version continues to occupy disk space.
- **What actually happens in the experiment:** The script creates a 1 GB parent file, then creates three child files. After that, three new versions(each of size 200 blocks) are generated, and all children are updated to point to the latest version. Before rebooting, the unused older versions still occupy disk space. After rebooting the system, remounting the file system, and performing the initial read of the parent file, those unused versions are cleaned up. As a result, the disk space corresponding to versions no longer referenced by any child is no longer shown as occupied.
- **How to run the experiment:** Run the script: `sudo ./test_eviction_before_reboot.sh`
  Measure the disk space usage (using `du --block-size=4096 evic_par_file`)
  Unmount the file system.
  Mount the file system again.
  Read the parent file once.
  Measure the disk space usage again.
- **Expected result:** The blocks corresponding to unused versions are deleted after the first read after the mount, reducing disk space usage.